

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΠΛΗΡΟΦΟΡΙΚΗΣ**



Τομέας: Υπολογιστική Υψηλών Επιδόσεων
Εργαστήριο: Εργαστήριο Πληροφοριακών Συστημάτων
Υψηλών Επιδόσεων

**Συγκριτική Ανάλυση και Πειραματική
Αξιολόγηση Συστημάτων Παράλληλου
Προγραμματισμού Βασισμένου σε
Εργασίες**

Διπλωματική Εργασία

του φοιτητή

Γεωργίου Τσάβου του Κωνσταντίνου

Αριθμός Μητρώου: 1084606

Επιβλέπων: Παναγιώτης Χατζηδούκας, Αναπληρωτής Καθηγητής,
Πανεπιστήμιο Πατρών

Πάτρα, Ιούνιος 2026

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα:

**Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες**

του φοιτητή του Τμήματος Μηχανικών Ηλεκτρονικών Υπολογιστών και
Πληροφορικής

Γεωργίου Τσάβου του Κωνσταντίνου

(Α.Μ.: 1084606)

παρουσιάστηκε δημόσια και εξετάστηκε επιτυχώς από την
τριμελή εξεταστική επιτροπή στις __/__/____.

Ο Επιβλέπων

Μέλος της Επιτροπής

Μέλος της Επιτροπής

Παναγιώτης

Χατζηδούκας

Αναπληρωτής Καθηγητής

Σπυρίδων

Σιούτας

Καθηγητής

Γεώργιος Παπαδημητρίου

Επίκουρος Καθηγητής

Στοιχεία Διπλωματικής Εργασίας

Θέμα:

Συγκριτική Ανάλυση και Πειραματική Αξιολόγηση
Συστημάτων Παράλληλου Προγραμματισμού Βασισμένου
σε Εργασίες

Φοιτητής:

Γεώργιος Τσάβος του Κωνσταντίνου

Ομάδα Επίβλεψης:

Παναγιώτης Χατζηδούκας

Αναπληρωτής Καθηγητής, Πανεπιστήμιο Πατρών

Εργαστήριο:

Εργαστήριο Πληροφοριακών Συστημάτων Υψηλών Επιδόσεων

Περίληψη

Λέξεις-κλειδιά: HPC, Διαργασίες, task-parallelization, heterogeneity

Abstract

Keywords: HPC, Διεργασίες, task-parallelization, heterogeneity

Ευχαριστίες

Περιεχόμενα

Συντομογραφίες	ix
Κατάλογος σχημάτων	x
Κατάλογος πινάκων	xi
1 Εισαγωγή	1
1.1 Σκοπός και Αντικείμενο της Εργασίας	1
1.2 Δομή της Εργασίας	2
2 Θεωρητικό Υπόβαθρο	5
2.1 Παραλληλοποίηση με Εργασίες	5
2.2 Χρονοδρομολόγηση Εργασιών	6
2.3 Αρχιτεκτονική NUMA	7
2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες	9
2.5 Παραλληλοποίηση στην Python	10
3 Το Σύστημα StarPU	13
3.1 Τι είναι το StarPU	13
3.2 Η Αρχιτεκτονική του StarPU	14
3.2.1 Οντότητες Εργασίας	14
3.2.2 Οντότητες Εκτέλεσης	15
Βιβλιογραφία	17

Συντομογραφίες

HPC	High Performance Computing
CPU	Central Processing Unit
PU	Processing Unit
GPU	Graphics Processing Unit
FPGA	Field-Programmable Gate Array
FCFS	First-Come-First-Served
RR	Round Robin
NUMA	Non-Uniform Memory Access
SMP	Symmetric Multi-Processing
UMA	Uniform Memory Access
HEFT	Heterogeneous Earliest Finish Time
GIL	Global Interpreter Lock
PEP	Python Enhancement Proposal
VSM	Virtual Shared Memory
MPI	Message Passing Interface

Κατάλογος σχημάτων

2.1	Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1]) . .	6
2.2	Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])	8
2.3	Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])	8

Κατάλογος πινάκων

Κεφάλαιο 1

Εισαγωγή

1.1 Σκοπός και Αντικείμενο της Εργασίας

Η ραγδαία εξέλιξη της υπολογιστικής επιστήμης και η ανάγκη για την ανάλυση όλο και μεγαλύτερου όγκου δεδομένων έχουν καταστήσει τον παράλληλο προγραμματισμό αναπόσπαστο κομμάτι της σύγχρονης έρευνας και τεχνολογίας. Καθώς η φυσική αύξηση της συχνότητας των επεξεργαστών (ρολόι) έχει φτάσει στα όριά της, η σύγχρονη υπολογιστική ισχύς πηγάζει πλέον κυρίως από αρχιτεκτονικές πολλαπλών πυρήνων (multi-core) και συστήματα συμμετρικής ή μη-συμμετρικής πολυεπεξεργασίας. Παράλληλα, γλώσσες υψηλού επιπέδου όπως η Python κυριαρχούν στους επιστημονικούς υπολογισμούς λόγω της ευκολίας χρήσης τους και του πλούσιου οικοσυστήματος βιβλιοθηκών τους, όμως η αποτελεσματική εκμετάλλευση του σύγχρονου υλικού (hardware) μέσω αυτής αποτελεί μια μείζονα πρόκληση, κυρίως λόγω εγγενών περιορισμών της γλώσσας στον ταυτόχρονο προγραμματισμό.

Αντικείμενο της παρούσας διπλωματικής εργασίας είναι η συγκριτική ανάλυση και πειραματική αξιολόγηση δύο διαφορετικών συστημάτων παράλληλου προγραμματισμού βασισμένου σε εργασίες (task-based parallel programming). Το μοντέλο αυτό προσφέρει ένα υψηλό επίπεδο αφαίρεσης, όπου το πρόγραμμα διασπάται σε ανεξάρτητες μονάδες εκτέλεσης, τις "εργασίες" (tasks). Ο προγραμματιστής ορίζει τις εργασίες και τις μεταξύ τους εξαρτήσεις, ενώ το σύστημα χρόνου εκτέλεσης (runtime system) αναλαμβάνει δυναμικά να τις δρομολογήσει στους διαθέσιμους πυρήνες, να διαχειριστεί τη μνήμη και να εξισορροπήσει τον φόρτο εργασίας, απαλλάσσοντας τον χρήστη από τον χειροκίνητο και επίπονο έλεγχο των νημάτων.

Σκοπός της εργασίας είναι να μελετηθεί πώς τα συστήματα αυτά ανταποκρίνονται στις σύγχρονες προκλήσεις του υλικού και του λογισμικού. Η πρώτη μεγάλη πρόκληση αφορά τις αρχιτεκτονικές **NUMA (Non-Uniform Memory Access)**, οι οποίες κυριαρχούν στα συστήματα υψηλών επιδόσεων (HPC). Σε αυτές, η μνήμη είναι χωρισμένη σε τοπικά τμήματα ανά επεξεργαστή, με αποτέλεσμα η πρόσβαση σε απομακρυσμένα τμήματα μνήμης να προκαλεί καθυστερήσεις και συμφόρηση. Τα task-based συστήματα πρέπει, επομένως, να έχουν επίγνωση της τοπολογίας (NUMA-awareness) για να διατηρούν την

τοπικότητα των δεδομένων. Η δεύτερη και σημαντικότερη πρόκληση στο περιβάλλον της Python είναι το **Global Interpreter Lock (GIL)**, ένας μηχανισμός που επιτρέπει την εκτέλεση κώδικα Python από ένα μόνο νήμα ανά πάσα στιγμή. Σε *cpu-bound* εφαρμογές, η χρήση απλών νημάτων οδηγεί σε *GIL thrashing* (διαρκείς εναλλαγές για την απόκτηση του lock), προκαλώντας κατακερματισμό της χρήσης της CPU (ακόμα και στο 1.5%) και ακυρώνοντας τον παραλληλισμό.

Για την αντιμετώπιση αυτών των προβλημάτων, η εργασία εξετάζει και συγκρίνει δύο πλατφόρμες με θεμελιωδώς διαφορετικές αρχιτεκτονικές φιλοσοφίες:

- **StarPU (και η διεπαφή StarPUPY):** Ένα σύστημα χαμηλού επιπέδου σε C, σχεδιασμένο για *shared memory* και ετερογενή συστήματα. Θα επικεντρωθούμε στην Python διεπαφή του, StarPUPY για την πειραματική αξιολόγηση.
- **torcpy:** Ένα Python-native σύστημα βασισμένο στο κατανεμημένο πρότυπο MPI (μέσω της `mpi4py`), ιδανικό για κατενεμημένα συστήματα.

Η πειραματική αξιολόγηση και η συνεισφορά της εργασίας εστιάζουν στην υλοποίηση και σύγκριση των δύο συστημάτων πάνω σε τρεις αντιπροσωπευτικές *cpu-bound* εφαρμογές: έναν αλγόριθμο βελτιστοποίησης (CMA-ES με επίλυση Poisson PDE), μια ροή επεξεργασίας εικόνας (Gabor Filters), και μια προσομοίωση Monte Carlo (Basket Options). Μέσα από αυτές τις εφαρμογές, η εργασία ποσοτικοποιεί τον χρόνο εκτέλεσης σε συνθήκες ισχυρού παραλληλισμού (64 πυρήνες), αναδεικνύοντας τα όρια και τις βέλτιστες πρακτικές χρήσης του κάθε συστήματος.

1.2 Δομή της Εργασίας

Η παρούσα διπλωματική εργασία είναι οργανωμένη σε επτά συνολικά κεφάλαια, ακολουθώντας μια λογική πορεία από τη θεωρητική θεμελίωση στην πρακτική εφαρμογή και την ανάλυση:

- **Κεφάλαιο 2 -- Θεωρητικό Υπόβαθρο:** Παρουσιάζεται το θεωρητικό υπόβαθρο γύρω από την παραλληλοποίηση με εργασίες, την ετερογένεια και τις προκλήσεις των αρχιτεκτονικών NUMA, την παραλληλοποίηση σε Python και το GIL.
- **Κεφάλαιο 3 -- Το Σύστημα StarPU:** Αναλύονται οι δυνατότητες και η αρχιτεκτονική του StarPU σε περιβάλλον C, καθώς και ο τρόπος διαχείρισης των πόρων.
- **Κεφάλαιο 4 -- Η Διεπαφή StarPUPY:** Εξετάζεται η μεταφορά του StarPU στην Python, οι μέθοδοι διαχείρισης του GIL και οι ιδιαιτερότητες του χρονοπρογραμματισμού του.

- **Κεφάλαιο 5 -- Το Σύστημα torcpy:** Αναλύεται το μοντέλο εκτέλεσης του torcpy, το API του, ο τρόπος δρομολόγησης των tasks και η χρήση του MPI για την απομόνωση διεργασιών.
- **Κεφάλαιο 6 -- Εφαρμογές και Μεθοδολογία:** Περιγράφεται αναλυτικά ο κώδικας και η μαθηματική δομή των τριών εφαρμογών (CMA-ES, Image Processing, Monte Carlo), καθώς και ο τρόπος λήψης των μετρήσεων.
- **Κεφάλαιο 7 -- Πειραματική Αξιολόγηση και Συμπεράσματα:** Παρουσιάζονται τα διαγράμματα και τα αποτελέσματα των μετρήσεων, αναλύονται τα φαινόμενα saturation και κλιμάκωσης, και συνοψίζονται τα τελικά συμπεράσματα της σύγκρισης.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Παραλληλοποίηση με Εργασίες

Στον χώρο του HPC, η παραλληλοποίηση με εργασίες (task parallelism) είναι μια προσέγγιση, η πλέον πιο διαδεδομένη, που επιτρέπει την εκτέλεση πολλαπλών εργασιών ταυτόχρονα, όπου κάθε εργασία μπορεί να είναι ανεξάρτητη ή να έχει εξαρτήσεις από άλλες εργασίες. Αυτή η μέθοδος είναι ιδιαίτερα χρήσιμη όταν οι εργασίες έχουν διαφορετικά μεγέθη ή απαιτούν διαφορετικούς πόρους. Αυτό έχει ως αποτέλεσμα την καλύτερη εκμετάλλευση των διαθέσιμων πόρων του συστήματος και τη μείωση του χρόνου εκτέλεσης.

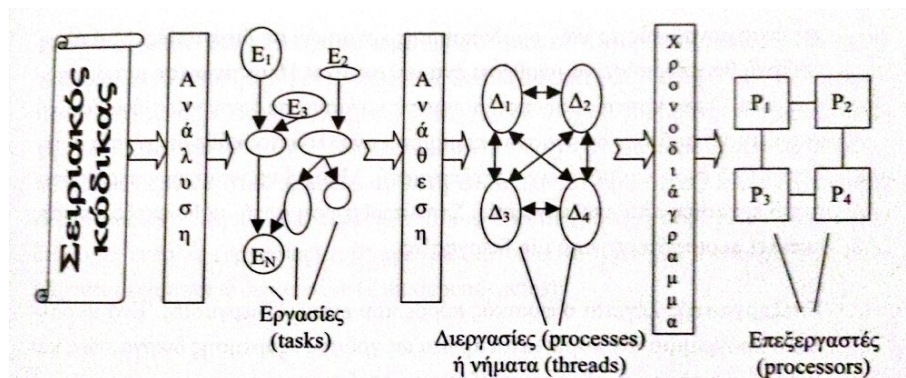
Η μετατροπή ενός αλγορίθμου για εκτέλεση σε ένα περιβάλλον παραλληλοποίησης με εργασίες απαιτεί διάφορα βήματα. Αρχικά, η υλοποίηση και η μελέτη του αλγορίθμου σε σειριακή μορφή σε μια γλώσσα προγραμματισμού θα βοηθήσει στην κατανόηση της λειτουργίας του και στην αναγνώριση των τμημάτων που μπορούν να παραλληλοποιηθούν. Ιδανικά αυτό θα θέλαμε να γίνεται αυτόματα από τον μεταγλωτιστή και το λειτουργικό σύστημα, αλλά στην πράξη απαιτείται η παρέμβαση του προγραμματιστή για να καθορίσει τις εργασίες και τις εξαρτήσεις τους [1].

Τα βήματα που πρέπει να λάβει υπόψη ο προγραμματιστής στην διαδικασία παραλληλοποίησης ενός αλγορίθμου, είναι τα εξής [1]:

1. **Ανάλυση του Αλγορίθμου σε Εργασίες (tasks):** Ο αλγόριθμος πρέπει να αναλυθεί για να εντοπιστούν τα τμήματα που μπορούν να εκτελεστούν ανεξάρτητα. Αυτό μπορεί να περιλαμβάνει την ανάλυση των βρόχων, των συναρτήσεων και των δεδομένων που χρησιμοποιούνται.
2. **Ανάθεση σε Διεργασίες (processes) ή Νήματα (threads):** Οι εργασίες πρέπει να ανατεθούν σε διαφορετικές διεργασίες (ή νήματα) ή να εκτελεστούν σε διαφορετικούς πόρους, με βάση την διαθεσιμότητα και τις απαιτήσεις του κάθε ενός.
3. **Χρονοδρομολόγηση των Διεργασιών (scheduling):** Η χρονοδρομολόγηση των εργασιών είναι ένα σύνθετο βήμα που αναλύεται στα εξής:

- (α') **Ανάθεση κάθε Διεργασίας σε ένα Πόρο:** Κάθε διεργασία πρέπει να ανατεθεί σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων.
- (β') **Απόφαση για την Εκτέλεση των Εργασιών:** Δηλαδή, πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε στο βήμα 3α'.

Τα παραπάνω βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες φαίνονται και στο παρακάτω Σχήμα 2.1.



Σχήμα 2.1: Βήματα παραλληλοποίησης ενός αλγορίθμου με εργασίες. (Πηγή: [1])

2.2 Χρονοδρομολόγηση Εργασιών

Όπως είδαμε στην προηγούμενη ενότητα, η χρονοδρομολόγηση των εργασιών είναι το πιο σύνθετο βήμα από τα υπόλοιπα, το οποίο στην σημερινή εποχή γίνεται όλο και πιο δύσκολο λόγω της αυξανόμενης πολυπλοκότητας των συστημάτων HPC και των εφαρμογών που εκτελούνται σε αυτά.

Η χρονοδρομολόγηση των εργασιών περιλαμβάνει την ανάθεση κάθε διεργασίας σε έναν πόρο, όπως ένας επεξεργαστής ή ένας πυρήνας, με βάση τις απαιτήσεις της εργασίας και τη διαθεσιμότητα των πόρων. Επιπλέον, πρέπει να ληφθεί υπόψη η απόφαση για το πότε θα εκτελεστεί κάθε εργασία (task) στον συγκεκριμένο πόρο που του αντιστοιχίστηκε. Αυτό καθιστά την επιλογή μιας αποτελεσματικής στρατηγικής (ή πολιτικής) χρονοδρομολόγησης κρίσιμη για την απόδοση των εφαρμογών σε περιβάλλοντα HPC.

Παλαιότερα ήταν αρκετό να γίνεται η χρήση απλών πολιτικών χρονοδρομολόγησης, όπως η First-Come-First-Served (FCFS) ή η Round Robin (RR), για να επιτευχθεί ικανοποιητική απόδοση. Ωστόσο, με την αύξηση της πολυπλοκότητας των συστημάτων και των εφαρμογών, αυτές οι απλές πολιτικές δεν είναι πλέον επαρκείς. Σήμερα, απαιτούνται πιο εξελιγμένες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη παράγοντες όπως οι

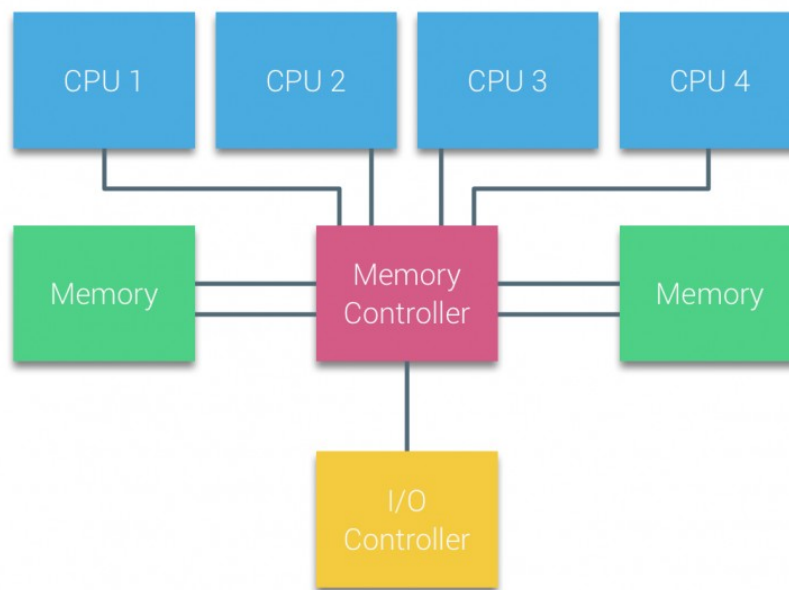
εξαρτήσεις μεταξύ των εργασιών, η εκτίμηση του χρόνου εκτέλεσης κάθε εργασίας, η δυναμική κατάσταση των πόρων και η τοπικότητα των δεδομένων κάθε εργασίας στην μνήμη, διότι πλέον τα συστήματα είναι πολύπολκα εισάγοντας έννοιες ετερογένειας (heterogeneity) και πολλαπλών επεξεργαστηκών μονάδων, όπως οι αρχιτεκτονικές **NUMA**.

Οπότε θέλουμε η χρονοδρομολόγηση των εργασιών να γίνεται δυναμικά είτε από το λειτουργικό σύστημα είτε από το περιβάλλον εκτέλεσης (runtime environment) με βάση τις τρέχουσες συνθήκες του συστήματος και των εργασιών. Αυτό μπορεί να επιτευχθεί μέσω της χρήσης τεχνικών όπως **Work Stealing**, **HEFT** κ.α. που λαμβάνουν υπόψη τους διάφορους παράγοντες που αναφέρθηκαν προηγουμένως. Τέτοιες τεχνικές υλοποιούνται όπως θα δουμε σε παρακάτω κεφάλαια και από τη **torc** και τη **StarPU** [2, 3] στο επίπεδο του περιβάλλοντος εκτέλεσης.

2.3 Αρχιτεκτονική NUMA

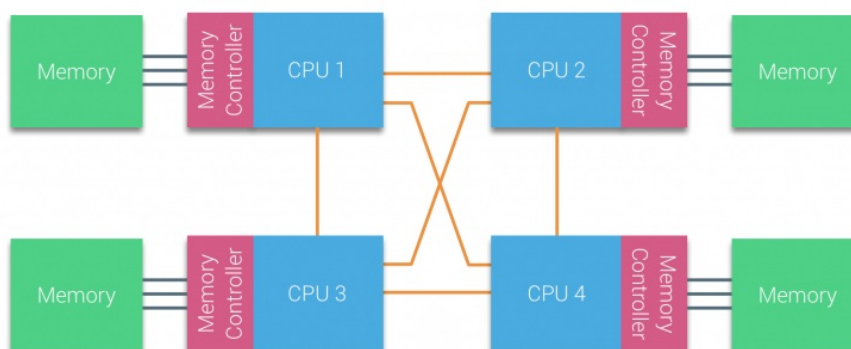
Όπως αναφέρθηκε και στην προηγούμενη ενότητα, η πολυπλοκότητα των συστημάτων **HPC** έχει αυξηθεί λόγω της διαρκούς εξέλιξης τους και ανάγκης για επίλυση πιο σύνθετων επιστημονικών και βιομηχανικών προβλημάτων. Ιστορικά, ωστόσο, το μεγαλύτερο εμπόδιο στην επίτευξη της μέγιστης δυνατής υπολογιστικής απόδοσης δεν ήταν η ταχύτητα των ίδιων των επεξεργαστών, αλλά ο περιορισμός που είναι ευρέως γνωστός ως το «εμπόδιο του Von Neumann» (Von Neumann bottleneck). Το φαινόμενο αυτό περιγράφει την εγγενή ανισορροπία μεταξύ της ταχύτητας επεξεργασίας των Κεντρικών Μονάδων Επεξεργασίας (**Central Processing Unit - CPU**) και της ταχύτητας μεταφοράς των δεδομένων από και προς την κύρια μνήμη του συστήματος, γεγονός που συχνά αφήνει τους επεξεργαστές σε κατάσταση αναμονής, στερημένους από τα απαραίτητα δεδομένα για την εκτέλεση των υπολογισμών [4].

Αρχικά, για να αντιμετωπιστή αυτό το πρόβλημα, οι σχεδιαστές συστημάτων υιοθέτησαν μια αρχιτεκτονική που ονομάζεται Ομοιόμορφη Πρόσβαση Μνήμης (**Uniform Memory Access - UMA**), η οποία είναι γνωστή και ως Συμμετρική Πολυεπεξεργασία (**Symmetric Multi-Processing - SMP**), όπου πολλοί επεξεργαστές μοιράζονται την ίδια φυσική μνήμη, όπως φαίνεται και στο Σχήμα 2.2. Το βασικό πλεονέκτημα αυτής της προσέγγισης ήταν η απλότητά της από πλευράς προγραμματισμού. Όλοι οι επεξεργαστές λάμβαναν απολύτως ισότιμη και ομοιόμορφη πρόσβαση (ίσο χρόνο απόκρισης) σε οποιαδήποτε θέση της μνήμης. Ωστόσο, καθώς ο αριθμός των επεξεργαστών σε ένα σύστημα αυξανόταν, η συνολική απόδοση του συστήματος άρχισε να υποφέρει λόγω της αυξημένης συμφόρησης στη μνήμη [5].



Σχήμα 2.2: Αρχιτεκτονική Ομοιόμορφης Πρόσβασης Μνήμης (UMA) ή Συμμετρικής Πολυεπεξεργασίας (SMP). (Πηγή: [5])

Για την επίλυση του παραπάνω προβλήματος αναπτύχθηκε μια νέα αρχιτεκτονική που ονομάζεται Μη Ομοιόμορφη Πρόσβαση Μνήμης (**Non-Uniform Memory Access - NUMA**), όπου κάθε επεξεργαστής έχει τη δική του τοπική μνήμη, αλλά μπορεί επίσης να έχει πρόσβαση στη μνήμη άλλων επεξεργαστών. Παράδειγμα αυτής της αρχιτεκτονικής φαίνεται στο Σχήμα 2.3. Σε αυτή την αρχιτεκτονική, η πρόσβαση στη τοπική μνήμη ενός επεξεργαστή είναι ταχύτερη από την πρόσβαση στη μνήμη άλλων επεξεργαστών, γεγονός που εισάγει μια μη ομοιόμορφη συμπεριφορά στην πρόσβαση στη μνήμη. Η αρχιτεκτονική **NUMA** επιτρέπει την καλύτερη κλιμάκωση των συστημάτων με πολλούς επεξεργαστές, αλλά απαιτεί επίσης πιο περίπλοκες στρατηγικές χρονοδρομολόγησης και διαχείρισης μνήμης για να μεγιστοποιηθεί η απόδοση των εφαρμογών που εκτελούνται σε αυτά τα συστήματα [5, 6].



Σχήμα 2.3: Αρχιτεκτονική Μη Ομοιόμορφης Πρόσβασης Μνήμης (NUMA). (Πηγή: [5])

2.4 Προκλήσεις της Αρχιτεκτονικής NUMA στην παραλληλοποίηση με Εργασίες

Η φαινομενική απλότητα που προσφέρει η λογική αφαίρεση του μοντέλου των εργασιών έρχεται σε άμεση σύγκρουση με την εξαιρετικά πολύπλοκη φυσική πραγματικότητα του υλικού **NUMA**. Η εγγενής απόκρυψη των αρχιτεκτονικών λεπτομερειών από τον προγραμματιστή σημαίνει ότι το σύστημα χρόνου εκτέλεσης (runtime system) καλείται να λάβει κρίσιμες αποφάσεις σχετικά με την τοποθέτηση δεδομένων και τη δρομολόγηση των εργασιών, χωρίς πάντα να διαθέτει την πλήρη εικόνα της υποκείμενης τοπολογίας. Όταν τα αυτόματα συστήματα δρομολόγησης κατανέμουν εργασίες χωρίς επίγνωση της φυσικής τοποθεσίας των δεδομένων (**NUMA-oblivious scheduling**), οι επιπτώσεις στην απόδοση είναι καταστροφικές. Αυτή η αναντιστοιχία οδηγεί σε εκθετική αύξηση της κίνησης στους διαύλους διασύνδεσης (interconnect traffic), ακραία υποβάθμιση του εύρους ζώνης και αδυναμία γραμμικής κλιμάκωσης, διαμορφώνοντας ένα τοπίο στο οποίο η συγκριτική ανάλυση και η πειραματική αξιολόγηση των σύγχρονων συστημάτων παράλληλου προγραμματισμού καθίσταται θεμελιώδους σημασίας για την ακαδημαϊκή έρευνα [7].

Προβλήματα όπως τα παραπάνω καλούνται να λύσουν διάφορες τεχνικές που έχουν αναπτυχθεί για την βελτιστοποίηση της απόδοσης σε συστήματα **NUMA**. Μία τέτοια τεχνική είναι η **NUMA-aware Work Stealing (NUMA-WS)**, η οποία τροποποίησε τον πυρήνα του Intel Cilk Plus εφαρμόζοντας ολιστικά την αρχή του **Work-First**¹. Αντί να βασίζεται σε τεχνικές τυχαίας κλοπής που οδηγούν σε μεταναστεύσεις μεγάλου κόστους, το **NUMA-WS** προτείνει τη μηχανική ώθησης (work pushing). Το σύστημα αναθέτει στον εργάτη την προσπάθεια τοποθέτησης ενός υπολογιστικού πλαισίου (full frame) ακριβώς δίπλα στα δεδομένα του, εκβιάζοντας τη δημιουργία τοπικότητας. Το **NUMA-WS** για να αποφύγει να επιβαρύνει πολύ το τρέχων νήμα-εργάτη εφαρμόζει ώθηση δεδομένων μόνο σε τρία συγκεκριμένα σενάρια: κατά την εκτέλεση πράξεων συγχρονισμού, κατά την επιστροφή θυγατρικού νήματος σε αδρανές γονικό, και μετά από επιτυχή κλοπή εργασίας. Με αυτόν τον τρόπο, σύμφωνα με τη φιλοσοφία **Work-First**, το κόστος της βελτίωσης της τοπικότητας **NUMA** δεν επιβαρύνει ποτέ το ενεργό νήμα-εργάτη, αλλά αποσβένεται αποκλειστικά στον αδρανή νήμα-κλέφτη. Το «κατώφλι ώθησης» εγγυάται ότι οι συνολικές λειτουργίες παραμένουν σταθερές και ελεγχόμενες, αποτρέποντας τεράστιο φόρτο εργασίας. Στην πράξη, τα αποτελέσματα είναι εντυπωσιακά: ο πληθωρισμός έργου **Work Inflation**² μπορεί να μειωθεί σημαντικά στο **NUMA-WS**, με σημαντική βελτίωση της κλιμάκωσης σε συστήματα με 32 και πλέον πυρήνες [7].

Παρόμοιες τεχνικές έχουν αναπτυχθεί και για άλλες πλατφόρμες, όπως η **StarPU** [3]

¹Η αρχή του **Work-First** υποστηρίζει ότι νέες εργασίες πρέπει να εκτελούνται το συντομότερο δυνατόν.

²Το **Work Inflation** αναφέρεται στο φαινόμενο όπου ο συνολικός χρόνος εκτέλεσης που παράγουν όλα τα νήματα μαζί είναι μεγαλύτερος από την εργασία που θα εκτελούσε ένας μόνο επεξεργαστής σειριακά.

και η **torcpy** [2] που θα μελετηθούν στην παρούσα εργασία, οι οποίες υλοποιούν διάφορες στρατηγικές χρονοδρομολόγησης που λαμβάνουν υπόψη την τοπολογία **NUMA** για να βελτιώσουν την απόδοση των εφαρμογών που εκτελούνται σε τέτοια συστήματα.

2.5 Παραλληλοποίηση στην Python

Η **Python** είναι μια από τις πιο δημοφιλείς υψηλού επιπέδου γλώσσες προγραμματισμού σήμερα, γνωστή για την απλότητα, την ευκολία χρήσης της και το πλούσιο οικοσύστημα βιβλιοθηκών της.

Στην καρδιά της **Python (CPython)** βρίσκεται το **Gloabl Interpreter Lock (GIL)**, το οποίο είναι ένας μηχανισμός που επιτρέπει μόνο σε ένα νήμα λειτουργικού συστήματος να εκτελεί **Python bytecode** τη φορά, ακόμα και σε συστήματα με πολλούς επεξεργαστές. Αυτό σημαίνει ότι η παραλληλοποίηση με νήματα (threading) στην **Python** δεν μπορεί να εκμεταλλευτεί πλήρως τις δυνατότητες των πολυπύρηνων συστημάτων, περιορίζοντας την απόδοση των εφαρμογών που απαιτούν έντονη επεξεργασία [8, 9, 10]. Πιο συγκεκριμένα:

- **CPU-bound Εφαρμογές:** Για υπολογιστικά εντατικές εργασίες, το GIL λειτουργεί ως απόλυτο σημείο συμφόρησης, ακυρώνοντας εντελώς τα πλεονεκτήματα των πολυπύρηνων επεξεργαστών **NUMA**.
- **I/O-bound Εφαρμογές:** Για εργασίες που εξαρτώνται από αιτήματα δικτύου, λειτουργίες αρχείων κλπ, το GIL είναι εξαιρετικά αποδοτικό (τα νήματα απελευθερώνουν το GIL όταν περιμένουν).

Ωστόσο, υπάρχουν διάφορες βιβλιοθήκες και εργαλεία που έχουν αναπτυχθεί για να παρακάμψουν τους περιορισμούς του GIL και να επιτρέψουν την παραλληλοποίηση στην **Python** [11]. Μερικά από αυτά περιλαμβάνουν:

- **Multiprocessing Module:** Η βιβλιοθήκη **multiprocessing** επιτρέπει τη δημιουργία νέων διεργασιών, οι οποίες μπορούν να εκτελούνται παράλληλα και να εκμεταλλεύονται πολλούς επεξεργαστές, παρακάμπτοντας έτσι το GIL. Ωστόσο, η χρήση διεργασιών μπορεί να είναι πιο βαριά σε πόρους και να απαιτεί περισσότερη μνήμη σε σύγκριση με τα νήματα.
- **Threading Module:** Η βιβλιοθήκη **threading** επιτρέπει τη δημιουργία νήματων, αλλά λόγω του GIL, δεν είναι αποτελεσματική για CPU-bound εργασίες.
- **Asyncio Module:** Η βιβλιοθήκη **asyncio** παρέχει υποστήριξη για ασύγχρονο προγραμματισμό, επιτρέποντας την εκτέλεση πολλαπλών εργασιών (tasks) ταυτόχρονα χωρίς να μπλοκάρει το κύριο νήμα, αλλά δεν είναι κατάλληλη για CPU-bound εργασίες.

- **Concurrent.futures Module:** Η βιβλιοθήκη **concurrent.futures** παρέχει μια υψηλού επιπέδου διεπαφή για την εκτέλεση εργασιών (tasks) σε νήματα ή διεργασίες, επιτρέποντας την εύκολη παραλληλοποίηση, αλλά εξακολουθεί να υπόκειται στους περιορισμούς του GIL.

Κεφάλαιο 3

Το Σύστημα StarPU

3.1 Τι είναι το StarPU

Όπως έχουμε ήδη αναφέρει πολλές φορές είναι ότι τα συστήματα που χρησιμοποιούνται για εφαρμογές **HPC**, είναι πολύ περίπλοκα. Γι' αυτό το λόγο άλλωστε έχουμε αρχιτεκτονικές **NUMA**, που εισάγουν την έννοια της ετερογένειας (**heterogeneity**) στην πρόσβαση στη μνήμη αλλά και στην φύση της επεξεργαστικής μονάδας (**Processing Unit - PU**). Δηλαδή ένα σύστημα NUMA μπορεί να έχει επεξεργαστικές μονάδες που είναι πέρα από **CPUs**, αλλά και **GPUs** ή **FPGAs** κ.α. Η ετερογένεια αυτή κάνει την ανάπτυξη εφαρμογών για τέτοια συστήματα πολύ δύσκολη, καθώς ο προγραμματιστής πρέπει να λάβει υπόψη του τις ιδιαιτερότητες και την υποκείμενη αρχιτεκτονική κάθε τύπου επεξεργαστικής μονάδας και να διαχειριστεί την κατανομή των εργασιών και των δεδομένων με τρόπο που να μεγιστοποιεί την απόδοση.

Την λύση σε αυτό το πρόβλημα έρχεται να δώσει το **StarPU**, ένα runtime system που παρέχει ένα ενοποιημένο περιβάλλον για την ανάπτυξη εφαρμογών σε ετερογενή συστήματα. Το StarPU επιτρέπει στους προγραμματιστές να γράφουν κώδικα που μπορεί να εκτελεστεί σε διάφορους τύπους επεξεργαστικών μονάδων χωρίς να χρειάζεται να ανησυχούν για τις λεπτομέρειες της αρχιτεκτονικής κάθε μονάδας. Παρέχει μηχανισμούς για την κατανομή των εργασιών, τη διαχείριση της μνήμης και την επικοινωνία μεταξύ των επεξεργαστικών μονάδων, επιτρέποντας έτσι την αποτελεσματική εκμετάλλευση των πόρων του συστήματος [3, 12].

Η θεμελιώδης αρχή λειτουργίας του StarPU είναι να «κάνει δυναμικά ό,τι δεν μπορεί πλέον να γίνει στατικά», καθιστώντας τις εφαρμογές φορητές και ιδιαίτερα κλιμακώσιμες χωρίς να θυσιάζεται η απόδοση χαμηλού επιπέδου [13]. Προσφέροντας ένα πλούσιο API σε C, C++ και Fortran, καθώς και υποστήριξη pragma OpenMP, το σύστημα εκτέλεσης απαλλάσσει τον προγραμματιστή από την αντιμετώπιση των λεπτομερειών υλικού χαμηλού επιπέδου, επιτρέποντάς του να επικεντρωθεί αποκλειστικά στην αλγοριθμική λογική [12].

Η παρούσα ανάλυση παρέχει μια ολοκληρωμένη και εξαντλητική παρουσίαση του τι επιτυγχάνει το StarPU και των περίπλοκων εσωτερικών μηχανισμών που χρησιμοποιεί για να το πετύχει. Η αξιολόγηση αναλύει τις βασικές αρχιτεκτονικές οντότητες, το υποσύστημα

Εικονικής Κοινής Μνήμης (**Virtual Shared Memory - VSM**), αλγορίθμους χρονοπρογραμματισμού εργασιών βασισμένους σε μοντέλα απόδοσης πραγματικού χρόνου, τον κύκλο εκτέλεσης των εσωτερικών εργασιών του συστήματος εκτέλεσης και την επέκταση αυτών των παραδειγμάτων σε περιβάλλοντα συστάδων κατανεμημένης μνήμης μέσω **MPI** [14, 15].

3.2 Η Αρχιτεκτονική του StarPU

Η αρχιτεκτονική του StarPU αποτελείται από διάφορα βασικά στοιχεία που συνεργάζονται για να παρέχουν το περιβάλλον εκτέλεσης για ετερογενείς εφαρμογές. Οι τρεις κύριες οντότητες είναι οι εξής: Οντότητες Εργασίας, Οντότητες Εκτέλεσης και Οντότητες Δεδομένων. Αυτός ο τριμερής διαχωρισμός επιτρέπει στο σύστημα χρόνου εκτέλεσης να αποσυνδέσει τον αλγοριθμικό ορισμό ενός προβλήματος από τη φυσική εκτέλεσή του στο υλικό [16].

3.2.1 Οντότητες Εργασίας

Στην καρδιά του προγραμματιστικού μοντέλου του StarPU βρίσκονται τα **Codelets** (struct starpu_codelet). Ένα codelet ενσωματώνει έναν υπολογιστικό αλγόριθμο (ή πυρήνα υπολογισμού) και παρέχει πολλαπλές υλοποιήσεις για τις διάφορες διαθέσιμες επεξεργαστικές μονάδες (**PU**s) με βάση την υποκείμενη αρχιτεκτονική. Για παράδειγμα, ένα codelet σχεδιασμένο για την κλιμάκωση ενός διανύσματος μπορεί να περιέχει έναν απλό δείκτη συνάρτησης στην C για την εκτέλεση σε CPU, μια υλοποίηση CUDA για GPU και μια υλοποίηση OpenCL για συσκευές AMD. Η δομή του codelet επισημαίνει ποιο υλικό (hardware) υποστηρίζει μέσω μιας μάσκας bit (π.χ. STARPU_CPU | STARPU_CUDA) και υποδεικνύει τον αριθμό των buffer δεδομένων που χειρίζεται (nbuffers). Ενοποιώντας τις διάφορες υλοποιήσεις σε ένα ενιαίο codelet, το StarPU επιτρέπει στο σύστημα εκτέλεσης να επιλέξει δυναμικά την καταλληλότερη υλοποίηση με βάση τη διαθεσιμότητα των πόρων και την τοπικότητα των δεδομένων [12, 16, 17].

Η δεύτερη οντότητα εργασίας είναι τα **Tasks**. Ένα task είναι μια συγκεκριμένη εμφάνιση ενός codelet που έχει δεσμευτεί με ένα συγκεκριμένο σύνολο δεδομένων. Ένα task λειτουργεί ως μεσολαβητής μεταξύ του συστήματος χρόνου εκτέλεσης (runtime system) και της εφαρμογής. Κατά την υποβολή ενός task μέσω των βοηθητικών συναρτήσεων starpu_task_submit ή starpu_task_insert, ο προγραμματιστής καθορίζει το codelet, τα δεδομένα πάνω στα οποία θα εκτελεστεί (μέσω **Data Handles**) και τους τρόπους πρόσβασης (π.χ. STARPU_R, STARPU_W, STARPU_RW). Επιπλέον, η δομή του task υποστηρίζει προαιρετικά πεδία που παρέχουν υποδείξεις προς τον χρονοπρογραμματιστή, όπως επίπεδα προτεραιότητας, χρώμα εκτέλεσης για προφίλ και συναρτήσεις επανάκλησης

(callback_func) που ενεργοποιούνται μετά την επιτυχή ολοκλήρωση του task. Τα Tasks είναι εκ φύσεως ασύγχρονα από προεπιλογή, ωστόσο οι προγραμματιστές μπορούν να επιβάλουν σύγχρονη εκτέλεση ορίζοντας task->synchronous = 1 [12, 16, 17].

Η τελευταία οντότητα εργασίας είναι τα **Jobs**. Ένα job είναι μια δομή που χρησιμοποιεί εσωτερικά το StarPU για να αναπαραστήσει ένα task. Είναι μια χαμηλού επιπέδου οντότητα που διασχίζει τον εσωτερικό γράφο χρονοπρογραμματισμού. Ο διαχωρισμός μεταξύ tasks και jobs δίνει τη δυνατότητα για εξελεγμένες λειτουργίες όπως η παράλληλη εκτέλεση εργασιών, όπου μια μόνη δομή job μπορεί να συντονίσει πολλαπλές δομές παράλληλων tasks αντιστοιχισμένες σε μια ομάδα εργατών που χρησιμοποιούν παραλληλοποίηση σε επίπεδο νημάτων (όπως OpenMP). Η δομή job περιέχει εσωτερικά μεταδεδομένα που απαιτούνται για την ανάλυση εξαρτήσεων, την ουρά προτεραιοτήτων και την παρακολούθηση μοντέλων απόδοσης [12, 16, 17].

3.2.2 Οντότητες Εκτέλεσης

Βιβλιογραφία

- [1] Παπαδάκης Στυλιανός και Διαμαντάρας Κωνσταντίνος. *Προγραμματισμός και Αρχιτεκτονική Συστημάτων Παράλληλης Επεξεργασίας*. 1η έκδοση. Αθήνα: Εκδόσεις Κλειδάριθμος, 2012. ISBN: 978-960-461-446-2.
- [2] Panagiotis E Hadjidoukas κ.ά. ``torepy: Supporting task parallelism in Python". Στο: *SoftwareX* 12 (2020), σ. 100517.
- [3] Cédric Augonnet κ.ά. ``StarPU: a unified platform for task scheduling on heterogeneous multicore architectures". Στο: *European Conference on Parallel Processing*. Springer. 2009, σσ. 863–874.
- [4] Wikipedia. *NUMA (Non-Uniform Memory Access)*. 2024. URL: https://en.wikipedia.org/wiki/Non-uniform_memory_access.
- [5] Frank Denneman. *NUMA Deep Dive Part 1: From UMA to NUMA*. 2016. URL: <https://frankdenneman.ai/2016-07-07-numa-deep-dive-part-1-uma-numa/>.
- [6] Super Micro Computer, Inc. *What Is Non-Uniform Memory Access (NUMA)?* 2025. URL: <https://www.supermicro.com/en/glossary/numa>.
- [7] Justin Deters κ.ά. ``A NUMA-Aware Provably-Efficient Task-Parallel Platform Based on the Work-First Principle". Στο: *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018, σσ. 59–70. DOI: [10.1109/IISWC.2018.8573486](https://doi.org/10.1109/IISWC.2018.8573486). URL: <http://dx.doi.org/10.1109/IISWC.2018.8573486>.
- [8] RapidFork Technology. *Breaking Down Python Concurrency: The Global Interpreter Lock (GIL) and Its Effect on Multi-threading*. 2023. URL: <https://rapidfork.medium.com/breaking-down-python-concurrency-the-gil-and-its-effect-on-multi-threading-b5ab99083cb2>.
- [9] Sam Gross. *PEP 703 -- Making the Global Interpreter Lock Optional in CPython*. Python Enhancement Proposal 703. Python Software Foundation, 2023. URL: <https://peps.python.org/pep-0703/>.

- [10] William Ruys κ.ά. "A deep dive into task-based parallelism in Python". Στο: *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE. 2024, σσ. 1147–1149.
- [11] A.P. Adewole κ.ά. "Understanding and Implementing Concurrency in Python-based Systems". Στο: *International Journal of Engineering Research & Technology (IJERT)* 15.03 (2026). ISSN: 2278-0181. URL: <https://www.ijert.org/understanding-and-implementing-concurrency-in-python-based-systems-ijertv15is031148>.
- [12] StarPU Development Team. *StarPU Handbook for StarPU 1.4.12*. INRIA. 2025. URL: <https://starpu.gitlabpages.inria.fr/doc.html>.
- [13] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: A Runtime System for Scheduling Tasks, or How to Get Portable Performance on Accelerator-Based Platforms Without the Agonizing Pain*. Presentation at NVIDIA GPU Technology Conference (GTC 2010), San Jose, USA. 2010. URL: https://www.nvidia.com/content/GTC-2010/pdfs/2160_GTC2010.pdf.
- [14] Cédric Augonnet, Samuel Thibault και Raymond Namyst. *StarPU: a Runtime System for Scheduling Tasks over Accelerator-Based Multicore Machines*. Research Report RR-7240. INRIA, Μαρ. 2010, σ. 33. URL: <https://inria.hal.science/inria-00467677>.
- [15] Cédric Augonnet κ.ά. "Data-aware task scheduling on multi-accelerator based platforms". Στο: *2010 IEEE 16th international conference on parallel and distributed systems*. IEEE. 2010, σσ. 291–298.
- [16] StarPU Development Team. *StarPU Developer Documentation*. INRIA. 2026. URL: https://files.inria.fr/starpu/doc/starpu_dev.pdf.
- [17] StarPU Development Team. *StarPU: A Unified Runtime System for Heterogeneous Multicore Architectures*. 2026. URL: <https://starpu.gitlabpages.inria.fr/>.

